> 更新日期: 2025-10-09 / 文档编写: LynxShu / 许可协议: CC-BY-NC-SA 4.0

SillyTavern 配置文件 (config.yaml) 详细说明文档

`config.yaml` 是 SillyTavern 的核心配置文件，它允许您对服务器、安全性、API 连接、性能等各个方面进行深度定制。本指南将详细解释该文件中的每个配置项，并提供实际应用示例。

> **注意**:
>
> - 修改此文件后，您需要**重启 SillyTavern 服务器 (终端)**才能使更改生效。
> - YAML 格式对缩进非常敏感，请确保您的修改遵循正确的缩进格式。
> - `true` 或 `false` 值不需要加引号。

1. 数据配置

- `dataRoot`: 定义用户数据（如聊天记录、角色卡）的存储根目录。默认值为 `./data`，即 SillyTavern 主目录下的 `data` 文件夹。

2. 服务器配置

- `listen`: 是否监听传入连接。如果你希望从其他设备（如手机、平板）访问 SillyTavern，必须将其设置为 `true`。默认为 `false`。
- `listenAddress`:
  - `ipv4`: 监听的 IPv4 地址。`0.0.0.0` 表示监听所有可用的网络接口，这是实现局域网访问的关键。
  - `ipv6`: 监听的 IPv6 地址。`[::]` 表示监听所有可用的 IPv6 网络接口。
- `protocol`:
  - `ipv4`: 是否启用 IPv4 协议。
  - `ipv6`: 是否启用 IPv6 协议。
  - `auto` 选项可以自动检测支持的协议。至少需要启用一个协议。
- `dnsPreferIPv6`: 是否优先使用 IPv6 进行 DNS 解析。仅在你的网络环境对 IPv6 支持良好时开启。
- `port`: 服务器运行的端口号。默认为 `8000`。

3. 浏览器启动配置

- `browserLaunch`:
  - `enabled`: 服务器启动时是否自动打开浏览器。
  - `browser`: 指定用于打开的浏览器。`default` 为系统默认浏览器，也可指定为 `firefox`、`chrome`、`edge` 等。
  - `hostname`: 在浏览器中打开的地址主机名。`auto` 会自动决定，你也可以强制指定为 `localhost` 或你的域名。
  - `port`: 在浏览器中打开的地址端口。`-1` 表示使用服务器设定的端口 (`port`)。
  - `avoidLocalhost`: 在 `auto` 模式下避免使用 `localhost` 作为主机名。

4. SSL 配置

- `ssl`:
  - `enabled`: 是否启用 HTTPS。启用后，您可以通过 `https://` 访问。
  - `certPath`: SSL 证书文件（.pem）的路径。
  - `keyPath`: SSL 密钥文件（.pem）的路径。

5. 安全配置

- `whitelistMode`: 是否开启白名单模式。开启后，只有在 `whitelist` 列表中的 IP 地址才能访问。默认为 `true`，强烈建议保持开启。
- `enableForwardedWhitelist`: 如果你使用了反向代理（如 Nginx），开启此项会检查 `X-Forwarded-For` / `X-Real-IP` 头中的 IP 地址。
- `whitelist`: 允许访问的 IP 地址列表。默认包含 `::1` (IPv6 本地回环) 和 `127.0.0.1` (IPv4 本地回环)。
- `whitelistDockerHosts`: 自动将 Docker 的主机和网关 IP 加入白名单。
- `basicAuthMode`: 是否为所有访问启用基础认证（即弹窗要求输入用户名和密码）。
- `basicAuthUser`:
  - `username`: 基础认证的用户名。
  - `password`: 基础认证的密码。
- `enableCorsProxy`: 是否启用 CORS 代理中间件。
- `requestProxy`:
  - `enabled`: 是否为所有传出的 HTTP/HTTPS 请求启用代理。这对于访问被屏蔽的 API（如 OpenAI）非常有用。
  - `url`: 代理服务器的地址。支持 http, https, socks, socks5, socks4, pac 等协议。格式示例: `socks5://user:pass@host:port`。
  - `bypass`: 不需要走代理的主机列表。
- `enableUserAccounts`: 启用多用户模式。
- `sessionTimeout`: 用户会话超时时间（秒）。`-1` 表示永不超时，`0` 表示浏览器关闭后超时，正数表示无活动 `N` 秒后超时。
- `disableCsrfProtection`: 禁用 CSRF 保护。**不推荐**。
- `securityOverride`: 禁用启动时的安全检查。**不推荐**。

6. 日志配置

- `logging`:
  - `enableAccessLog`: 记录所有访问请求到 `access.log` 文件和控制台。
  - `minLogLevel`: 控制台显示的最低日志级别。`0` (DEBUG) 最详细，`3` (ERROR) 最简洁。

7. 速率限制配置

- `rateLimiting`:
  - `preferRealIpHeader`: 在反向代理环境下，使用 `X-Real-IP` 头进行速率限制。

8. 备份配置

- `backups`:
  - `common`:
    - `numberOfBackups`: 为每个聊天和设置文件保留的备份数量。
  - `chat`:
    - `enabled`: 是否启用聊天自动备份。
    - `maxTotalBackups`: 每个用户最多保留的聊天备份总数。`-1` 为不限制。
    - `throttleInterval`: 聊天备份的节流间隔（毫秒），防止过于频繁的备份。

9. 缩略图配置

- `thumbnails`:
  - `enabled`: 是否启用缩略图生成。
  - `format`: 缩略图格式 (`jpg` 或 `png`)。
  - `quality`: `jpg` 格式的图像质量 (0-100)。
  - `dimensions`: 不同类型（背景、头像）缩略图的最大尺寸。

10. 性能相关配置

- `performance`:
  - `lazyLoadCharacters`: 懒加载角色卡。当你有大量角色卡时，能显著提升初始加载性能。
  - `memoryCacheCapacity`: 解析后的角色卡可占用的最大内存。`0` 为禁用。
  - `useDiskCache`: 为角色卡启用磁盘缓存，提升性能。

11. 缓存清除配置

- `cacheBuster`:
  - `enabled`: 首次加载或上传图片后清除浏览器缓存。**重要提示**: 此功能需要 `localhost` 或 HTTPS 域名才能工作。

12. API 与扩展配置

- `allowKeysExposure`: 是否允许通过 API 暴露密钥。
- `whitelistImportDomains`: 允许从中导入角色卡的域名白名单。
- `extensions`:
  - `enabled`: 是否启用 UI 扩展。
  - `autoUpdate`: 版本更新时是否自动更新扩展。

    - `models`: HuggingFace 模型的配置，如自动下载和具体的模型 ID。
- `enableDownloadableTokenizers`: 是否允许按需下载额外的分词器。

13. 各 API 服务商特定配置

- `openai`:
    - `randomizeUserId`: 是否向 OpenAI API 发送随机用户 ID。
    - `captionSystemPrompt`: 为图像描述功能添加的系统提示词。
- `deepl`:
    - `formality`: DeepL 翻译的形式（正式或非正式）。
- `mistral`:
    - `enablePrefix`: 是否用对话历史中的最后一条助理消息预填充回复。
- `ollama`:
    - `keepAlive`: 控制模型在请求后保留在内存中的时间。`-1` (无限), `0` (立即卸载), `N` (保留 N 秒)。
    - `batchSize`: 生成请求的批处理大小。
- `claude`:
    - `enableSystemPromptCache`: 是否启用系统提示词缓存。**请注意**：仅当聊天历史之前的提示部分（如角色设定、世界信息等）是**静态不变**的时候才应开启，否则会因为缓存未命中而浪费费用。
    - `cachingAtDepth`: 在指定深度缓存消息历史。**请谨慎使用**。应设置为一个非负整数（例如 `0`），代表要缓存的消息深度。任何其他值都将被忽略。此功能行为可能无法预测，需自行承担风险。
    - `extendedTTL`: 是否将缓存的生存时间（TTL）从默认的5分钟延长到1小时。**请注意**：延长 TTL 会增加成本（1小时 TTL 的价格是基础价格的2倍）。
- `gemini`:
    - `apiVersion`: 使用的 Gemini API 端点版本。根据配置文件注释，有效选项为 `'v1beta'` 或 `'v1alpha'`。

14. 服务器插件配置

- `enableServerPlugins`: 是否启用服务器插件。
- `enableServerPluginsAutoUpdate`: 启动时是否自动更新服务器插件。

实际应用实例

实例一：开放局域网访问

目标：你希望在电脑上运行 SillyTavern，然后用同一网络下的手机或平板来访问。

操作步骤:

1. 找到 `SERVER CONFIGURATION` 部分。
2. 将 `listen` 的值从 `false` 改为 `true`。
3. 确认 `listenAddress` 下的 `ipv4` 的值为 `0.0.0.0`。

4. 找到 `SECURITY CONFIGURATION` 部分。
5. 在 `whitelist` 列表下，添加你手机或平板的局域网 IP 地址。

修改示例：

```
# -- SERVER CONFIGURATION --
# Listen for incoming connections
listen: true # <--- 从 false 改为 true
# Listen on a specific address, supports IPv4 and IPv6
listenAddress:
  ipv4: 0.0.0.0 # <--- 确保是这个值
  ipv6: '[::]'

# ... 中间省略 ...

# -- SECURITY CONFIGURATION --
# Toggle whitelist mode
whitelistMode: true
# Whitelist of allowed IP addresses
whitelist:
  - ::1
  - 127.0.0.1
  - 192.168.1.101 # <--- 在这里添加你手机的局域网 IP
  - 192.168.0.0/16 # <--- 这样修改是允许所有 192.168 网段的 IP
```

说明：重启服务后，你就可以通过 `http://<你电脑的IP地址>:8000` 在手机上访问了。

实例二：修改访问端口号

目标：默认的 `8000` 端口被其他程序占用了，需要更换一个新端口。

操作步骤：

1. 找到 `SERVER CONFIGURATION` 部分。
2. 修改 `port` 的值为你想要的新端口号，例如 `8888`。

修改示例：

```
# Server port
port: 8888 # <--- 从 8000 修改为 8888
# -- SSL options --
```

说明：重启服务后，你的访问地址将变为 `http://localhost:8888`。

实例三：为 API 请求设置代理

目标：由于网络原因，无法直接连接 OpenAI 或其他 AI 服务的 API，需要通过代理服务器进行访问。

操作步骤：

1. 找到 `REQUEST PROXY CONFIGURATION` 部分。
2. 将 `enabled` 的值从 `false` 改为 `true`。

3. 在 `url` 处填入你的代理服务器地址。

修改示例：

```
# -- REQUEST PROXY CONFIGURATION --
requestProxy:
  enabled: true # <--- 从 false 改为 true
  url: "http://127.0.0.1:7890" # <--- 示例: 使用本地的 HTTP 代理
  # bypass:
  #   - localhost
  #   - 127.0.0.1
```

说明：重启服务后，SillyTavern 发送给 AI 提供商的所有网络请求都会通过你设置的代理进行。

实例四：开启密码访问（基础认证）

目标：为你的 SillyTavern 增加一层简单的密码保护，防止未授权访问。

操作步骤：

1. 找到 `SECURITY CONFIGURATION` 部分。
2. 将 `basicAuthMode` 的值从 `false` 改为 `true`。
3. 在 `basicAuthUser` 下，修改 `username` 和 `password` 为你想要的凭据。

修改示例：

```
# -- SECURITY CONFIGURATION --
# ...
# Toggle basic authentication for endpoints
basicAuthMode: true # <--- 从 false 改为 true
# Basic authentication credentials
basicAuthUser:
  username: "user"      # <--- 修改为你想要的用户名
  password: "password" # <--- 修改为你想要的密码
# ...
```

说明：重启服务后，任何访问 SillyTavern 页面的用户都会看到一个弹窗，必须输入正确的用户名和密码才能进入。

实例五：优化 Ollama 连接性能

目标：如果你在本地使用 Ollama，希望模型一直加载在内存中以获得更快的响应速度。

操作步骤：

1. 滚动到文件末尾，找到 `OLLAMA API CONFIGURATION` 部分。
2. 确保 `keepAlive` 的值是 `-1`。

修改示例：

```yaml
# -- OLLAMA API CONFIGURATION --
ollama:
  keepAlive: -1 # <--- 设置为 -1，表示永久加载
  batchSize: -1
```

说明：设置为 `-1` 可以让模型在第一次被调用后就常驻在 VRAM/RAM 中，后续的对话响应会非常迅速。但这会持续占用你的硬件资源，请根据你的电脑配置决定。